

It is also possible to write methods that update the 'virtual slots' *position-rho* and *position-theta*:

```
> (defmethod-setf position-rho ((pos x-y-position)) (rho)
  (let* ((r (position-rho pos))
        (ratio (/ rho r)))
    (setf (position-x pos) (* ratio (position-x pos)))
    (setf (position-y pos) (* ratio (position-y pos)))))

> (defmethod-setf position-theta ((pos x-y-position)) (theta)
  (let ((rho (position-rho pos)))
    (setf (position-x pos) (* rho (cos theta)))
    (setf (position-y pos) (* rho (sin theta)))))
```

To update the ρ -coordinate you may write:

```
> (setf (position-rho pos) new-rho)
```

This is precisely the same syntax that would be used if the positions were explicitly stored as polar coordinates.

Class re-definition

The Common Lisp Object System provides a powerful class-redefinition facility.

When a *defclass* form is evaluated, and a class with the given name already exists, the existing class is redefined. Redefining a class modifies the existing class object to reflect the new class definition.

You may define methods on the generic function *class-changed* to control the class redefinition process. This generic function is invoked automatically by the system after *defclass* has been used to redefine an existing class; for example, suppose it becomes apparent that the application that requires representing positions uses polar coordinates more than it uses rectangular coordinates. It might make sense to define a sub-class of *position* that uses polar coordinates:

```
> (defclass rho-theta-position (position)
  ((rho :initform 0)
   (theta :initform 0))
  (:accessor-prefix position-))
```

The instances of *x-y-position* can be automatically updated by defining a *class-changed* method:

```
> (defmethod class-changed ((old x-y-position)
  (new rho-theta-position))
  ;; Copy the position information from old to new to make new
  ;; be a rho-theta-position at the same position as old.
  (let ((x (position-x old))
        (y (position-y old)))
    (setf (position-rho new) (sqrt (+ (* x x) (* y y)))
          (position-theta new) (atan y x)))
```

At this point, we can change an instance of the class *x-y-position*, *p1*, to be an instance of *rho-theta-position* by using *change-class*:

```
> (change-class p1 'rho-theta-position)
```

Inheritance

Inheritance is the key to program modularity within CLOS. A typical object-oriented program consists of several classes, each of which defines some aspect of behaviour. New classes are defined by including the appropriate classes as super-classes, thus gathering the desired aspects of behaviour into one class.

In general, slot descriptions are inherited by sub-classes. That is, slots defined by a class are usually slots implicitly defined by any sub-class of that class, unless the sub-class explicitly shadows the slot definition. A class can also shadow some of the slot options declared in the *defclass* form of one of its super-classes by providing its own description for that slot.

A sub-class inherits methods in the sense that any method applicable to an instance of a class is also applicable to instances of any sub-class of that class (all other arguments to the method being the same).

The inheritance of methods acts the same way regardless of whether the method was created by using *defmethod* or by using one of the *defclass* options that cause methods to be generated automatically.

I hope with this article I have managed to convince OO programmers that Lisp is generous enough to cater to your style of thinking. Stick with me, and I promise that you won't be disappointed. So far we've seen how to fit the nuts and bolts into the engine. Next month, we'll learn how to paint it a nice shiny red...I am referring to Graphical Programming in Lisp! 

References

- Successful Lisp, David B. Lamkins
- Let Over Lambda, Doug Hoyte
- CLOS: Integrating Object-Oriented and Functional programming, Richard P. Gabriel, Jon L. White, Daniel G. Bobrow

By: Vivek Shangari

The author is a hard-core hacker, if ever there was one. He always thinks programs, and holds a piece of code in his head all the time. His favourite past time is to stop random strangers on the street and start talking about the benefits of open source over proprietary software, till they agree to switch sides, or threaten to jump off the nearest building. Every now and then, he manages to talk a bunch of unsuspecting potential programmers into letting him train them in Lisp. It's not rare to find people stumble out after the training session 'zombified', or with tears running down their faces. "Those are tears of joy!" he will patiently explain. For those of you out there who cannot be deterred, you can reach him at vivek@codeCraft.in.